

CBflow Flow User Guide

CBflow Development Team

April 2026

CBflow Flow User Guide

1. Getting Started

1.1 Initialize a Workspace

1.2 Configure Your Run

1.3 Create and Run

2. Supported Flows

2.1 Flow Overview

2.2 Stage Dependency Diagrams

3. Flow Details

3.1 SYNTH — Logic Synthesis

3.2 FP — Floorplanning

3.3 PNR — Place and Route

3.4 FCT — Static Timing Analysis

3.5 LEC — Logic Equivalence Checking

3.6 EMIR — EM/IR Drop and Thermal Analysis

3.7 PHYV — Physical Verification

3.8 ECO — Engineering Change Orders

3.9 CLP — Conformal Low Power Verification

3.10 POPT — Power Optimization

3.11 FCFP — Fullchip Floorplanning

4. Running Flows

4.1 Complete Workflow (Step by Step)

4.2 Running Individual Stages

- 4.3 Re-running Stages
- 4.4 Viewing Status
- 4.5 Viewing Logs
- 4.6 Validation
- 5. Merged Flows
 - 5.1 SYNTH_FP (Synthesis + Floorplan)
 - 5.2 Other Merged Combinations
- 6. LSF Submission
 - 6.1 Submitting to LSF
 - 6.2 Queue Recommendations
 - 6.3 Checking LSF Status
- 7. Node and Branch Management
 - 7.1 Adding Custom Nodes
 - 7.2 Branches
 - 7.3 Viewing the Graph
- 8. Run Directory Structure
- 9. Configuration Reference
 - 9.1 Input Variable Naming Convention
 - 9.2 Project Phases
 - 9.3 Environment Variables
- 10. Typical PD Workflow
- 11. Quick Reference

CBflow Flow User Guide

Version: 2.0.0 | April 2026

1. Getting Started

1.1 Initialize a Workspace

Every flow run starts with a workspace. A workspace ties together a project, flow type, and design block.

```
cd /path/to/my/designs
mkdir pnr_workspace && cd pnr_workspace

cbflow workspace init \
  --project ravendrive \
  --flow PNR \
  --block cpu_core
```

This generates three files:

File	Purpose
<code>.cbflow.env</code>	Shell environment (project, release, paths, component versions)
<code>.cbflow.tcl</code>	Same environment in TCL format
<code>user_config.tcl</code>	Template you edit before creating a run

The workspace locks the **project**, **flow type**, and **block**. These cannot be changed after init.

1.2 Configure Your Run

Edit `user_config.tcl` with your design-specific paths:

```
# — Workspace-locked (do not modify)

set project(name)      "ravendrive"
set project(phase)     "P0"
set project(technology) "gf_22nm"
set design(name)       "cpu_core"
```

```

set design(top_module)    "cpu_core"
set flow(type)            "PNR"

# — User-editable
_____

set flow(run_name)        "run1"
set flow(test_mode)       false

# — Input paths
_____

set pnr(input,netlist)     "/data/synth_output/cpu_core.v"
set pnr(input,sdc_func_file) "/data/constraints/cpu_core.sdc"
set pnr(input,def_file)    "/data/floorplan/cpu_core.def"
set pnr(input,upf_file)    "/data/power/cpu_core.upf"

```

Test mode: Set `flow(test_mode) true` to run the entire flow without invoking EDA tools. CBflow will execute all stages, create directories and stamps, and display command file contents instead of running tools. Useful for validating your configuration before a real run.

1.3 Create and Run

```

# Create the run directory
cbflow workspace create --config user_config.tcl

# Enter the run directory and execute
cd P0_run_PNR_run1
make all

```

That's it. CBflow creates the run directory with a Makefile, stage-specific `.mk` files, and environment files, then `make all` executes every stage in dependency order.

2. Supported Flows

CBflow supports 11 physical design flows. Each flow has a defined set of stages that execute sequentially (or in parallel where dependencies allow).

2.1 Flow Overview

Flow	Tool	Vendor	Stages	Description
SYNTH	Fusion Compiler	Synopsys	4	RTL to gate-level netlist
FP	Fusion Compiler	Synopsys	6	Floorplanning and power grid
PNR	Fusion Compiler	Synopsys	9	Full place-and-route
FCT	PrimeTime	Synopsys	3	Static timing analysis
LEC	Formality	Synopsys	4	Logic equivalence checking
EMIR	RedHawk	Synopsys	4	EM/IR drop and thermal
PHYV	IC Validator	Synopsys	7	DRC, LVS, ERC, PERC
ECO	Fusion Compiler	Synopsys	3	Engineering change orders
CLP	VC LP	Synopsys	4	Low-power intent verification
POPT	PrimeTime	Synopsys	5	Power optimization
FCFP	Fusion Compiler	Synopsys	6	Fullchip floorplanning

2.2 Stage Dependency Diagrams

Each box is a stage. Arrows show execution order.

SYNTH



FP

```
inputs —> import_design —> floorplan —> powerplan —>  
export_data —> release_data
```

PNR

```
inputs1 —> place1 —> cts1 —> cts_opt1 —> route1 —> pro1 —>  
signoff1 —> export_data1 —> release_data1
```

FCT

```
inputs —> extraction —> timing
```

LEC

```
inputs —> setup —> compare —> analyze
```

EMIR

```
inputs —> power_analysis —> ir_drop —> thermal_analysis
```

PHYV (parallel verification)

```
           ┌─> drc ─┐  
           └─> lvs ─┘  
inputs ───┤          └─> merge_data —> release_data  
           └─> erc ─┘  
           └─> perc ─┘
```

ECO

```
inputs —> eco —> export_db
```

CLP

```
inputs —> clp —> export_data —> release_data
```

POPT

```
inputs —> merge_timing —> power_opt —> post_merge —>  
release_data
```

FCFP

```
inputs —> fc_floorplan —> fc_powerplan —> fc_post_floorplan —>  
export_data —> release_data
```

3. Flow Details

3.1 SYNTH — Logic Synthesis

Transforms RTL source code into an optimized gate-level netlist.

Stages and Subnodes:

Stage	Subnodes	What Happens
inputs1	rtl, sdc, library, upf, validate, finish	Load RTL files, timing constraints, cell libraries, power intent
synthesis1	setup, run, validate, finish	Elaborate design, map to gates, optimize for

Stage	Subnodes	What Happens
		timing/area/power
export_data1	setup, run, validate, finish	Export synthesized netlist (.v), constraints (.sdc), database
release_data1	setup, run, validate, finish	Package outputs for downstream flows

User Config — Input Variables:

```
set synth(input,rtl_files)      "/path/to/rtl_filelist.f"
set synth(input,sdc_func_file)  "/path/to/constraints.sdc"
set synth(input,upf_file)      "/path/to/power_intent.upf"
```

Key Outputs: - `results/netlist/synth.v` — Synthesized netlist - `results/sdc/synth.sdc` — Output constraints - `reports/synthesis/synth_qor.rpt` — Quality of results - `reports/synthesis/synth_timing.rpt` — Timing report - `reports/synthesis/synth_area.rpt` — Area report - `reports/synthesis/synth_power.rpt` — Power report

Make Targets:

```
make all          # Complete flow
make inputs1      # Load inputs only
make synthesis1   # Run synthesis (requires inputs1)
make export_data1 # Export results
```

3.2 FP — Floorplanning

Creates the physical floorplan with block placement and power grid.

Stages and Subnodes:

Stage	Subnodes	What Happens
inputs	netlist, sdc, def, upf, library, validate, finish	Load synthesized netlist, constraints, floorplan seed
import_design	setup, run, validate, finish	Import design into Fusion Compiler
floorplan	setup, run, validate, finish	Define die area, place macros, create blockages
powerplan	setup, run, validate, finish	Create power grid, stripes, rails
export_data	setup, run, validate, finish	Export DEF, save database
release_data	setup, run, validate, finish	Package floorplan deliverables

User Config — Input Variables:

```

set fp(input,netlist)      "/path/to/synth_netlist.v"
set fp(input,sdc_func_file) "/path/to/constraints.sdc"
set fp(input,def_file)     "/path/to/seed_floorplan.def"
set fp(input,upf_file)     "/path/to/power_intent.upf"

```

3.3 PNR — Place and Route

Complete physical implementation from placement through sign-off.

Stages and Subnodes:

Stage	Subnodes	What Happens
inputs1	netlist, sdc, def, upf, library, validate, finish	Load design data from FP output
place1	setup, run, validate, finish	Standard cell placement
cts1	setup, run, validate, finish	Clock tree synthesis
cts_opt1	setup, run, validate, finish	Post-CTS optimization
route1	setup, run, validate, finish	Global and detail routing
pro1	setup, run, validate, finish	Post-route optimization
signoff1	setup, run, validate, finish	Final timing sign-off
export_data1	setup, run, validate, finish	Export GDS,

Stage	Subnodes	What Happens
		netlist, SPEF
release_data1	setup, run, validate, finish	Package for tapeout

User Config — Input Variables:

```
set pnr(input,netlist)      "/path/to/fp_netlist.v"
set pnr(input,sdc_func_file) "/path/to/constraints.sdc"
set pnr(input,def_file)    "/path/to/floorplan.def"
set pnr(input,upf_file)    "/path/to/power_intent.upf"
```

Make Targets:

```
make all          # Complete PNR flow (9 stages)
make place1       # Run placement only
make cts1         # Run CTS (requires place1)
make route1       # Run routing (requires cts_opt1)
make status       # Show which stages are done
make retrace NODE=cts1 # Re-run from CTS onwards
```

3.4 FCT — Static Timing Analysis

Sign-off timing verification using Synopsys PrimeTime.

Stages and Subnodes:

Stage	Subnodes	What Happens
inputs	setup, netlist, sdc, spef, library, validate, finish	Load netlist, constraints, parasitics

Stage	Subnodes	What Happens
extraction	setup, run, validate, finish	Parasitic extraction, generate SPEF
timing	setup, run, validate, finish	Setup/hold analysis, violations, QoR

User Config — Input Variables:

```
set fct(input,netlist)          "/path/to/pnr_netlist.v"
set fct(input,sdc_func_file)    "/path/to/constraints.sdc"
set fct(input,spef)            "/path/to/parasitics.spef"
```

Key Outputs: - `results/extraction/parasitic.spef` — Extracted parasitics - `results/timing/timing.rpt` — Final timing report - `results/timing/violations.rpt` — Timing violations - `reports/timing/setup_timing.rpt` — Setup analysis - `reports/timing/hold_timing.rpt` — Hold analysis

3.5 LEC — Logic Equivalence Checking

Formal verification that two netlists are logically equivalent.

Stages and Subnodes:

Stage	Subnodes	What Happens
inputs	setup, netlist_golden, netlist_revised, constraints, validate, finish	Load golden (reference) and revised (implementation) netlists
setup	setup, run, validate, finish	Configure Formality, read

Stage	Subnodes	What Happens
		designs, set verification mode
compare	setup, run, validate, finish	Match compare points, run formal verification
analyze	setup, run, validate, finish	Analyze failing points, generate equivalence report

User Config — Input Variables:

```
set lec(input,netlist_golden)  "/path/to/golden_netlist.v"
set lec(input,netlist_revised) "/path/to/revised_netlist.v"
set lec(input,constraints)     "/path/to/
    formality_constraints.svf"
```

Key Outputs: - results/lec/comparison.rpt — Comparison results - results/lec/equivalence.rpt — Equivalence analysis - results/lec/analysis.log — Detailed analysis log - reports/lec/failing_points.rpt — Failing compare points

3.6 EMIR — EM/IR Drop and Thermal Analysis

Power integrity and thermal verification using Synopsys RedHawk.

Stages and Subnodes:

Stage	Subnodes	What Happens
inputs	setup, netlist, def, spef, library, validate, finish	Load design with parasitics

Stage	Subnodes	What Happens
power_analysis	setup, run, validate, finish	Static and dynamic power consumption
ir_drop	setup, run, validate, finish	Static/dynamic IR drop, voltage maps
thermal_analysis	setup, run, validate, finish	Thermal simulation, hotspot identification

User Config — Input Variables:

```
set emir(input,netlist) "/path/to/pnr_netlist.v"
set emir(input,def)     "/path/to/routed.def"
set emir(input,spef)    "/path/to/parasitics.spef"
```

Key Outputs: - results/emir/power_analysis.rpt — Power report - results/emir/ir_drop.rpt — IR drop analysis - results/emir/thermal_analysis.rpt — Thermal report - reports/emir/ir_drop/static_ir.rpt — Static IR drop - reports/emir/thermal/hotspots.rpt — Thermal hotspots

3.7 PHYV — Physical Verification

DRC, LVS, ERC, and PERC checks using Synopsys IC Validator.

Stages and Subnodes:

Stage	Subnodes	What Happens
inputs	setup, netlist, def, gds, validate, finish	Load GDS layout and source netlist
drc	setup, run, validate, finish	Spacing, width, density, via checks
lvs	setup, run, validate, finish	Layout extraction, schematic comparison
erc	setup, run, validate, finish	Power/ground connectivity, floating nets
perc	setup, run, validate, finish	Reliability, ESD, latch-up checks
merge_data	setup, run, validate, finish	Merge all verification results into summary
release_data	setup, run, validate, finish	Package reports for release

DRC, LVS, ERC, and PERC all depend only on inputs, so they can run in parallel.

User Config — Input Variables:

```
set phyv(input,netlist) "/path/to/pnr_netlist.v"
set phyv(input,def)     "/path/to/routed.def"
set phyv(input,gds)     "/path/to/layout.gds"
```

Key Outputs: - `results/drc/drc_results.rpt` — DRC violations - `results/lvs/lvs_results.rpt` — LVS comparison - `results/erc/erc_results.rpt` — ERC violations - `results/perc/perc_results.rpt` — PERC results - `results/phyv/phyv_summary.rpt` — Merged summary

3.8 ECO — Engineering Change Orders

Implement late-stage design changes using Synopsys Fusion Compiler.

Stages and Subnodes:

Stage	Subnodes	What Happens
inputs	setup, netlist, def, sdc, library, validate, finish	Load current design + ECO change list
eco	setup, run, validate, finish	Analyze impact, implement changes, verify
export_db	setup, run, validate, finish	Export modified netlist and database

User Config — Input Variables:

```
set eco(input,netlist)      "/path/to/current_netlist.v"
set eco(input,def)          "/path/to/current.def"
set eco(input,sdc_func_file) "/path/to/constraints.sdc"
set eco(input,eco_script)   "/path/to/eco_changes.tcl"
```

3.9 CLP — Conformal Low Power Verification

Verifies UPF/CPF power intent correctness using Synopsys VC LP.

Stages and Subnodes:

Stage	Subnodes	What Happens
inputs	setup, netlist, upf, power_spec, validate, finish	Load netlist and UPF/CPF power intent
clp	setup, run, validate, finish	Connectivity, isolation, level shifter, retention, power switch checks
export_data	setup, run, validate, finish	Export verification database and reports
release_data	setup, run, validate, finish	Package for release

User Config — Input Variables:

```

set clp(input,netlist)      "/path/to/netlist.v"
set clp(input,upf_file)    "/path/to/power_intent.upf"
set clp(input,power_spec)  "/path/to/power_spec.tcl"

```

Key Outputs: - `results/clp/power_verification.rpt` — Verification summary - `results/db/clp_verification.db` — Verification database - `reports/clp/connectivity_check.rpt` — Domain connectivity - `reports/clp/isolation_check.rpt` — Isolation rules - `reports/clp/level_shifter_check.rpt` — Level shifter rules - `reports/clp/retention_check.rpt` — Retention rules

3.10 POPT — Power Optimization

Post-synthesis power optimization using Synopsys PrimeTime.

Stages and Subnodes:

Stage	Subnodes	What Happens
inputs	setup, netlist, sdc, upf, validate, finish	Load design with power constraints
merge_timing	setup, run, validate, finish	Merge timing data for optimization
power_opt	setup, run, validate, finish	Clock gating, multi-Vt, power reduction
post_merge	setup, run, validate, finish	Post-optimization validation

Stage	Subnodes	What Happens
release_data	setup, run, validate, finish	Package optimized design

3.11 FCFP — Fullchip Floorplanning

Chip-level floorplan with macro placement and power planning using Synopsys Fusion Compiler.

Stages and Subnodes:

Stage	Subnodes	What Happens
inputs	setup, netlist, sdc, def, upf, library, validate, finish	Load chip-level design data
fc_floorplan	setup, run, validate, finish	Chip die, core utilization, macro placement, blockages
fc_powerplan	setup, run, validate, finish	Power grid, stripes, rail connections
fc_post_floorplan	setup, run, validate, finish	Congestion and timing optimization
export_data	setup, run, validate, finish	Export DEF, database
release_data	setup, run, validate, finish	Package deliverables

4. Running Flows

4.1 Complete Workflow (Step by Step)

```
# 1. Initialize workspace
cbflow workspace init --project ravendrive --flow PNR --block
    cpu_core

# 2. Edit user_config.tcl with your input paths

# 3. Create a run
cbflow workspace create --config user_config.tcl

# 4. Enter run directory
cd P0_run_PNR_run1

# 5. Run complete flow
make all

# 6. Check results
make status
ls reports/
ls results/
```

4.2 Running Individual Stages

Every stage is a Make target:

```
make inputs1      # Only load inputs
make place1       # Only run placement (inputs1 must be done)
make cts1         # Only run CTS (place1 must be done)
```

If a dependency stage hasn't run, Make will run it automatically.

4.3 Re-running Stages

CBflow tracks completion with stamp files in `.stamps/`. To re-run a stage:

```
# Option 1: Delete the stamp and re-run
rm .stamps/place1.stamp
make place1

# Option 2: Retrace from a stage (deletes that stamp + all
#           downstream)
make retrace NODE=cts1
make all      # Re-runs from cts1 onwards

# Option 3: Delete all stamps to force full re-run
make retrace
make all
```

4.4 Viewing Status

```
# Quick overview
make status

# Detailed view with subnodes
cbflow run status --details

# Dependency graph
cbflow run show-graph

# Check stamp files directly
ls -la .stamps/
```

4.5 Viewing Logs

```
# List all log files
cbflow run logs --list
```

```
# Tail recent output
cbflow run logs --tail 50

# Filter by severity
cbflow run logs --tail 100 --level ERROR
cbflow run logs --tail 100 --level WARNING
```

4.6 Validation

```
# Validate everything (config + inputs + outputs)
cbflow run validate

# Config only
cbflow run validate --type config

# Specific stage outputs
cbflow run validate --stage place1
```

5. Merged Flows

CBflow can combine multiple flows into a single run. Stage names are prefixed with the flow name.

5.1 SYNTH_FP (Synthesis + Floorplan)

```
cbflow workspace init --project ravendrive --flow SYNTH_FP --
    block cpu_core
```

Generates stages: `synth_inputs`, `synth_synthesis`, `synth_export_data`, ..., `fp_inputs`, `fp_import_design`, `fp_floorplan`, ...

The FP inputs stage automatically depends on the SYNTH export_data stage.

User Config for Merged Flow:

```

set flow(type) "SYNTH_FP"

# SYNTH inputs
set synth(input,rtl_files)      "/path/to/rtl.v"
set synth(input,sdc_func_file)  "/path/to/constraints.sdc"

# FP inputs (fed from SYNTH output)
set fp(input,netlist)           "/path/to/synth_netlist.v"
set fp(input,sdc_func_file)     "/path/to/constraints.sdc"
set fp(input,def_file)          "/path/to/seed.def"

```

5.2 Other Merged Combinations

- `SYNTH_FP_PNR` — Full front-to-back implementation
 - `SYNTH_FP` — Synthesis through floorplan
 - Any combination of sequential flows
-

6. LSF Submission

6.1 Submitting to LSF

```

# Submit complete flow
cbflow run all --lsf

# With specific queue
cbflow run all --lsf --queue XL

# Submit single stage
cbflow run stage --name route1 --lsf

```

6.2 Queue Recommendations

CBflow assigns queues based on the flow and stage:

Queue	Memory	CPUs	Time	Used For
S	8 GB	4	2h	inputs, validate, export, release stages
M	16 GB	8	4h	synthesis, floorplan, CTS, ECO, CLP
L	32 GB	16	8h	placement, routing opt, DRC, LVS, extraction, IR drop
XL	64 GB	32	12h	complex routing, thermal analysis
ultra	128 GB	64	24h	full-chip signoff

Detailed mappings by flow:

Flow	S	M	L	XL
SYNTH	inputs, export, release	synthesis		
PNR	inputs, release	export	place, cts, cts_opt, pro, signoff	route
FCT	inputs		extraction, timing	

Flow	S	M	L	XL
LEC	inputs, analyze	setup, compare		
EMIR	inputs		power_analysis, ir_drop	thermal_analysis
PHYV	inputs, merge, release	erc, perc	drc, lvs	
ECO	inputs	export_db	eco	
CLP	inputs, export, release	clp		
POPT	inputs, release	merge_timing, post_merge	power_opt	
FCFP	inputs, export, release	fc_post_floorplan	fc_floorplan, fc_powerplan	

6.3 Checking LSF Status

```
cbflow run lsf-status
```

7. Node and Branch Management

7.1 Adding Custom Nodes

Add custom stages to a run after creation:

```
# Add an ECO node after signoff
cbflow run add-node --node eco1 --type eco --dep signoff1
```

```
# List all nodes
cbflow run list-nodes

# Delete a custom node
cbflow run delete-node --node eco1
```

7.2 Branches

Create experimental branches from any stage:

```
# Branch from CTS for an experiment
cbflow run create-branch --name timing_experiment --from cts1

# List branches
cbflow run list-branches

# Delete branch
cbflow run delete-branch --node timing_experiment
```

7.3 Viewing the Graph

```
# Simple view
cbflow run show-graph

# With subnodes
cbflow run show-graph --detail
```

8. Run Directory Structure

After `make all` completes, a run directory contains:

```
P0_run_PNR_run1/
├── .stamps/                               Completion tracking
│   └── inputs1.stamp
```

```
|   └─ inputs1_netlist.stamp
|   └─ inputs1_sdc.stamp
|   └─ place1.stamp
|   └─ place1_setup.stamp
|   └─ place1_run.stamp
|   └─ ...
└─ .make/                               Stage makefiles
    └─ inputs1.mk
    └─ place1.mk
    └─ ...
└─ work/PNR/                           Tool working area
    └─ inputs/
        └─ netlist/
        └─ sdc/
        └─ ...
    └─ place/
        └─ setup/
        └─ run/
    └─ ...
└─ reports/                             Generated reports
    └─ timing/
        └─ setup_timing.rpt
        └─ hold_timing.rpt
└─ results/                             Output data
    └─ netlist/
    └─ sdc/
    └─ db/
└─ logs/                                Execution logs
└─ setup/
    └─ user_config.tcl                  Your configuration
└─ Makefile                            Auto-generated orchestrator
└─ .run.cbflow.env                     Shell environment
└─ .run.cbflow.tcl                     TCL environment
└─ .run.cnflow.tcl                     TCL native environment
```

9. Configuration Reference

9.1 Input Variable Naming Convention

Each flow uses a consistent naming pattern for input variables:

```
{flow_array}(input,{input_type})
```

Flow	Array	Common Input Variables
SYNTH	<code>synth()</code>	<code>synth(input,rtl_files)</code> , <code>synth(input,sdc_func_file)</code> , <code>synth(input,upf_file)</code>
FP	<code>fp()</code>	<code>fp(input,netlist)</code> , <code>fp(input,sdc_func_file)</code> , <code>fp(input,def_file)</code> , <code>fp(input,upf_file)</code>
PNR	<code>pnr()</code>	<code>pnr(input,netlist)</code> , <code>pnr(input,sdc_func_file)</code> , <code>pnr(input,def_file)</code> , <code>pnr(input,upf_file)</code>
FCT	<code>fct()</code>	<code>fct(input,netlist)</code> , <code>fct(input,sdc_func_file)</code> , <code>fct(input,spef)</code>
LEC	<code>lec()</code>	<code>lec(input,netlist_golden)</code> , <code>lec(input,netlist_revised)</code> , <code>lec(input,constraints)</code>
EMIR	<code>emir()</code>	<code>emir(input,netlist)</code> , <code>emir(input,def)</code> , <code>emir(input,spef)</code>
PHYV	<code>phyv()</code>	<code>phyv(input,netlist)</code> , <code>phyv(input,def)</code> , <code>phyv(input,gds)</code>
ECO	<code>eco()</code>	

Flow	Array	Common Input Variables
		<code>eco(input,netlist)</code> , <code>eco(input,def)</code> , <code>eco(input,sdc_func_file)</code> , <code>eco(input,eco_script)</code>
CLP	<code>clp()</code>	<code>clp(input,netlist)</code> , <code>clp(input,upf_file)</code> , <code>clp(input,power_spec)</code>
POPT	<code>popt()</code>	<code>popt(input,netlist)</code> , <code>popt(input,sdc_func_file)</code> , <code>popt(input,upf_file)</code>
FCFP	<code>fcfp()</code>	<code>fcfp(input,netlist)</code> , <code>fcfp(input,sdc_func_file)</code> , <code>fcfp(input,def_file)</code> , <code>fcfp(input,upf_file)</code>

9.2 Project Phases

Phase	Description	Typical Activities
P0	Initial implementation	First synthesis, initial floorplan, early timing
P1	Design refinement	Timing closure iterations, power optimization
P2	Final implementation	Sign-off timing, final DRC/LVS, ECO implementation
P3	Production release	Tapeout data generation, manufacturing release

9.3 Environment Variables

These are set automatically in `.run.cbflow.env` and `.run.cbflow.tcl`:

Variable	Description
<code>CBFLOW_RUN_DIR</code>	Absolute path to current run directory
<code>CBFLOW_FLOW_TYPE</code>	Flow type (e.g., PNR, SYNTH)
<code>CBFLOW_DESIGN_NAME</code>	Design/block name
<code>CBFLOW_RUN_NAME</code>	Run identifier (e.g., run1)
<code>CBFLOW_PROJECT_PHASE</code>	Phase (P0, P1, P2, P3)
<code>CBFLOW_PROJECT_NAME</code>	Project name
<code>CBFLOW_BLOCK_NAME</code>	Block name
<code>FLOW_DIR</code>	CBflow core directory
<code>CONFIG_ROOT</code>	Configuration root
<code>FLOW_CONFIG_VERSION</code>	Active flow config version
<code>UTILITIES_VERSION</code>	Utilities version

10. Typical PD Workflow

A typical chip design flow using CBflow follows this sequence:

Step 1: SYNTH

RTL --> gate-level netlist

Output: synth.v, synth.sdc

Step 2: FP

netlist --> floorplan with power grid

Output: floorplan.def

Step 3: PNR

floorplan --> placed & routed design

Output: routed.def, routed.gds, parasitics.spef

Step 4: FCT

routed design + parasitics --> timing sign-off

Output: timing.rpt, violations.rpt

Step 5: LEC

RTL vs synthesized netlist --> equivalence proof

synthesized vs P&R netlist --> equivalence proof

Output: equivalence.rpt

Step 6: PHYV

GDS --> DRC/LVS/ERC/PERC clean

Output: drc_results.rpt, lvs_results.rpt

Step 7: EMIR

routed design + parasitics --> IR drop / thermal pass

Output: ir_drop.rpt, thermal_analysis.rpt

Step 8: CLP

netlist + UPF --> low-power intent verified

Output: power_verification.rpt

Each step is an independent CBflow workspace + run. Outputs from one flow become inputs to the next.

11. Quick Reference

— WORKSPACE

```
cbflow workspace init -p ravendrive -t PNR -b cpu_core #  
Initialize
```

```
cbflow workspace create --config user_config.tcl # Create  
run
```

```
cbflow workspace list-runs          # List runs
cbflow workspace status             # Overview

# — RUN (from inside run directory)


---


make all                           # Run complete flow
make <stage>                       # Run single stage
make status                        # Check progress
make retrace NODE=<stage>          # Re-run from stage
make show_graph                   # Show dependency graph
make clean                         # Delete work/logs/stamps

# — FLOW INFO


---


cbflow flow types                  # List all 11 flows
cbflow flow info --flow PNR        # Show flow details
cbflow flow stages --flow SYNTH    # List stages

# — LOGS & DEBUG


---


cbflow run logs --list             # List logs
cbflow run logs --tail 50          # Recent output
cbflow run logs --level ERROR      # Errors only
cbflow run validate                # Validate config + outputs
```